

Countersign

A gate in front of every state-changing tool call a coding agent makes.

It refuses the edit when the evidence the task is holding no longer describes the repository. The verdict is a hash comparison, the refusal prints both digests so it can be checked rather than believed, and a forged receipt fails the replay.

,

What I set out to prevent

An agent reads a file, decides what to change, and writes several minutes later. In those minutes a formatter rewrites the file, a second agent edits the same tree, or a pull moves the branch the path sat on.

The write then lands and the tool call reports success. Nothing raises an error, because the file exists, the diff applies cleanly, and nothing conflicted. The repository ends up holding an edit derived from text that was never true of it.

Existing tooling answers the easy question, whether the call succeeded. I wanted the harder one answered before the call runs: is the evidence this task holds still an accurate description of the repository it is about to change?

Bob's own behaviour widens the gap

After an edit, the runtime tells the agent this, captured from a live payload:

```
"You do not need to re-read the file, as you have seen all changes.
Proceed with the task using these changes as the new
baseline."
```

The agent is told to treat a change as its new baseline, which is correct when nothing else moved and wrong when something did.

Bob's documentation is candid about the consequence: compaction is lossy, wrong context persists in the transcript, and the documented recovery is for a person to notice and start a new task. That leaves a human doing the job of a mechanism.

The rule I built the whole thing around

No state-changing action may use repository evidence whose identity is no longer current.

So the gate records, per task, what the task actually knew:

files it read

the SHA-256 digest of the file at the moment it was read

commands it ran

the digest of the result text the runtime returned

where the path sat

the committed revision, resolved through git

How it decides

I kept a model out of the decision path. The verdict is a SHA-256 comparison, so the same inputs always give the same answer.

exit 0

ADMITTED

the evidence a task holds still describes the repository

exit 2

REFUSED

the call does not run, and the refusal names which fact moved

Eight reasons it can refuse, each with its own code:

NO_MANIFEST the task holds no manifest

OUTSIDE_WORKSPACE the target is outside the workspace

UNVERIFIED_TARGET the path was never read

EVIDENCE_SUPERSEDED the file moved after the read

REVISION_MOVED committed at a different revision

CROSS_TASK_EVIDENCE the record belongs to another task

UNSUPPORTED_SUBTASK_EVIDENCE evidence from another subtask

COMMAND_RESULT_CHANGED a recorded command now contradicts

What a refusal actually says

```
REFUSED: EVIDENCE_SUPERSEDED About to modify pricing.ts. Evidence this task holds: baa5252515ea. Digest on disk now: f24af5254d22.
```

```
EVIDENCE_SUPERSEDED: baa5252515ea → f24af5254d22 (the file changed after this task observed it)
```

```
REVISION_MOVED: a512df3add44 → 2522bc69c806 (this path was committed at a different revision after the task observed it)  
Required recovery:
```

1. open a fresh task
2. re-observe pricing.ts
3. rerun the affected commands
4. record the new evidence manifest Refusal receipt: 582b05e62d75bb15b36dcc508822961aea3895dac43a3dffffbd203af87d4454e The action did not happen.

Both digests are printed, so the refusal is checkable rather than a matter of trust. The agent gets the four steps that clear it.

Every verdict leaves a receipt

A verdict you cannot check afterwards is an assertion. So each decision writes a receipt holding the inputs it used, both digests, the code, the reason and the time it took.

chained

TAMPER-EVIDENT BY ARITHMETIC

each receipt stores the hash of the one before it, so the log is walkable

replay

RE-DERIVED, NOT TRUSTED

countersign replay recomputes every verdict from the inputs its own receipt recorded

A machine that never ran the gate can still check that a verdict was reached rather than typed. On every push, the CI workflow replays a committed store and then forges a verdict to prove the replay has teeth.

How I wired it into Bob

Bob exposes lifecycle hooks, and the gate is driven by three of them.

SessionStart	session-start	opens the evidence manifest for the task
PostToolUse	record	digests every file read and every command result
PreToolUse	check	admits or refuses the proposed state-changing call

The block goes into `.bob/settings.json` for one workspace or `~/.bob/settings/settings.json` machine-wide, using Bob's documented schema. One installer merges it and resolves the workspace from the `cwd` field that every payload carries, so there is no per-project configuration.

One real Bob task, with the gate installed

1.05

BOBCOINS SPENT

a real task, not a rehearsal

44.6s

WALL TIME

eight tool calls

4

ATTEMPTS REFUSED

across two edit tools

0

BYTES CHANGED

the file was left untouched

Task eea2941f4db4a1f67c401695b74446d0. The agent read the file, then tried the edit four times. Every attempt came back exit 2 with EVIDENCE_SUPERSEDED and wrote its own receipt, chained to the one before it.

The agent's own closing report, unprompted:

`"The safety guard correctly blocked the edit... I need to re-read the current state before modifying it."`

The console reads the runtime, never a snapshot

Five pages, each its own URL, and every panel is filled by the store the gate itself wrote. There is no bundled sample data.

protect

connect a repository, install the hook, run one turn and watch the verdict

interceptor

every call with its class, verdict, exit code and latency

evidence

what each session held, and how many calls it lost

receipts

the chain, and a replay that recomputes each verdict

With the runtime down, each page says disconnected and prints the exact command to start it, rather than quietly rendering a wrong-shaped fallback. The button that runs an agent turn spawns Bob headlessly and reports the exit code Bob really returned.

What it costs, because a gate that slows the agent down gets uninstalled

308 ms

ONE DECISION

against a manifest holding 200 observations

10 s

THE HOOK TIMEOUT

Bob's default budget for a pre-tool hook

~30x

HEADROOM

measured over 20 runs on a two-core laptop

Reading the manifest and measuring the workspace is the whole cost. Nothing is called over the network and nothing is inferred, so the number does not move with repository size the way a model call would.

What it does not claim

Not a content check. The pending diff reaches the hook and the gate ignores it on purpose. A wrong or hostile edit to a file that has not moved is admitted. This enforces whether the facts are current, not whether the change is good.

A fetch it never saw is invisible. A local gate cannot compare against a branch that was never fetched. One recorded run shows the write admitted before the fetch and refused after it.

Enforcement is per machine. The hook stops the edit where it is installed. What travels between machines is the receipt, which the replay can check.

The store is chained, not signed. Hash chaining makes edits detectable and the replay makes verdicts re-derivable. Neither is a signature.

Organisation-wide enforcement as the default is not built, and a repository guarded on one machine is not guarded on another unless the hook is installed there too.

Where to look

Repository	public, and linked in the submission form
Verify the record	countersign-eight.vercel.app/record
Demo video	youtu.be/NQ43DN8eA8I
Product site	countersign-eight.vercel.app

The repository carries the gate, the local runtime, the console, the hook blocks and the captured payloads from a real session. 531 tests pass in three suites, the acceptance run drives 21 steps against a live runtime, and the CI workflow that replays committed stores is green.

Everything on these slides comes from a recorded run, not from a plan.